Guía de Contribución y Arquitectura del Proyecto

📌 Proyecto de Desarrollo de una Plataforma de Workflow para Revisión Jurídica (SaaS)

Empresa Patrocinadora: PluriOne S.A. de C.V. (Develop Talent & Technology)
Institución Académica: Instituto Tecnológico Superior de Atlixco (ITSA) — Residencia Profesional
Metodología: Scrum / Ágil
Documento Rector de Requerimientos: `Plantilla Historias Usuario.xls (1).xlsx` (17 Historias de Usuario Oficiales, HU-01 a HU-17)
Estado Actual: 🏁 **Etapa Terminal / Listo para Entrega (100% Completado)**
URL de Producción: https://legaltech-api-LegalprojectIA-1134.azurewebsites.net

📊 Matriz de Historias de Usuario y Progreso del Proyecto

Todas las **17 Historias de Usuario oficiales** han sido implementadas, verificadas mediante pruebas automáticas y desplegadas en la nube:

ID	ROL	FUNCIONALIDAD REQUERIDA	MÓDULO / ARCHIVOS CLAVE	ESTADO ACTUAL	% AVANCE
HU-01	Usuario Jurídico	Gestionar el flujo de revisión de documentos legales	`app/api/v1/endpoints/solicitudes.py` `app/models/solicitud.py` `app/templates/new_document.html`	🟢 Completado	100%
HU-02	Usuario Jurídico	Consultar bandeja de trabajo (Inbox de pendientes)	`app/api/v1/endpoints/workflows.py` `app/templates/tasks.html`	🟢 Completado	100%
HU-03	Usuario Jurídico	Asignación automática de revisores por balanceo de carga	`app/services/workflow_engine.py` `app/api/v1/endpoints/solicitudes.py`	🟢 Completado	100%
HU-04	Usuario Jurídico	Consultar documentos asignados para revisión	`app/api/v1/endpoints/solicitudes.py` `app/templates/documents.html` `app/templates/document_detail.html`	🟢 Completado	100%
HU-05	Usuario Jurídico	Revisar documentos legales dentro de la plataforma	`app/api/v1/endpoints/reviews.py` `app/templates/legal_review.html`	🟢 Completado	100%
HU-06	Usuario Jurídico	Agregar comentarios e hilos de discusión en la revisión	`app/models/comment.py` `app/api/v1/endpoints/reviews.py` `app/templates/comments.html`	🟢 Completado	100%
HU-07	Usuario Jurídico	Consultar observaciones y comentarios del documento	`app/templates/comments.html` `app/api/v1/endpoints/reviews.py`	🟢 Completado	100%
HU-08	Usuario Jurídico	Mantener control de versiones inmutables (v1, v2...)	`app/models/version.py` `app/services/workflow_engine.py` `app/api/v1/endpoints/documents.py`	🟢 Completado	100%
HU-09	Usuario Jurídico	Consultar y comparar versiones del documento	`app/templates/version_history.html` `app/api/v1/endpoints/documents.py`	🟢 Completado	100%
HU-10	Usuario Jurídico	Gestionar dictámenes y aprobaciones formales	`app/api/v1/endpoints/signatures.py` `app/templates/approval.html` `app/api/v1/endpoints/solicitudes.py`	🟢 Completado	100%
HU-11	Usuario Jurídico	Registrar firma electrónica con evidencia criptográfica	`app/models/extras.py` (`Firma`) `app/templates/signature.html` `app/api/v1/endpoints/signatures.py`	🟢 Completado	100%
HU-12	Auditor / Jurídico	Consultar la trazabilidad y auditoría completa	`app/core/audit_helper.py` `app/models/audit.py` `app/templates/traceability.html` `app/api/v1/endpoints/audit.py`	🟢 Completado	100%

ID	ROL	FUNCIONALIDAD REQUERIDA	MÓDULO / ARCHIVOS CLAVE	ESTADO ACTUAL	% AVANCE
HU-13	Usuario Jurídico	Recibir alertas y notificaciones de plazos y SLA	`app/models/extras.py` (`Notificacion`) `app/api/v1/endpoints/notifications.py`	🟢 Completado	100%

HU-14	Usuario Jurídico	Asistencia y análisis con Inteligencia Artificial	`app/services/azure_openai.py`
			`app/services/azure_doc_intel.py`
			`app/services/document_extractor.py`
			`app/api/v1/endpoints/ai_legal.py`
				🟢 Completado	100%

HU-15	Administrador	Acceder y operar el panel administrativo integral	`app/admin.py` (SQLAdmin)
			`app/templates/admin_panel.html`
			`app/api/v1/endpoints/users.py`
				🟢 Completado	100%

HU-16	Administrador	Gestionar y configurar flujos de trabajo dinámicos	`app/models/workflow.py`
			`app/services/workflow_engine.py`
			`app/api/v1/endpoints/workflows.py`
				🟢 Completado	100%

HU-17	Usuario Jurídico	Integración y almacenamiento con repositorio documental	`app/services/azure_blob.py` `app/api/v1/endpoints/documents.py`	🟢 Completado	100%

> 📌 Para la descripción analítica y los criterios de aceptación formales de cada historia, consulte la Matriz de Historias de Usuario.

🛠️ Stack Tecnológico Oficial

CAPA / DOMINIO	TECNOLOGÍA	PROPÓSITO
Backend Core	Python 3.11+ / FastAPI / Uvicorn	API RESTful asíncrona de alto rendimiento y arquitectura en capas
Persistencia Relacional	PostgreSQL 16 (SQLAlchemy 2.0 + Alembic)	Almacenamiento transaccional, ORM moderno y control de versiones de base de datos
Capa de Presentación Web	Jinja2 Templates + HTML5/CSS3 + TailwindCSS	Vistas web corporativas servidas por FastAPI, responsivas y sin dependencias pesadas
Identidad Corporativa	Microsoft Entra ID (MSAL.js local + OAuth2 / OIDC)	Single Sign-On (SSO), auto-aprovisionamiento y RBAC estricto
Criptografía Aplicada	Cryptography (AES-256-GCM / SHA-256)	Cifrado en reposo para datos clasificados y sellado de integridad de versiones
Inteligencia Artificial	Azure OpenAI (`gpt-5-mini` / `gpt-4o-mini`)	Extracción de riesgos, análisis de cláusulas y dictamen legal asistido
Procesamiento de Documentos	PyMuPDF + Azure AI Document Intelligence	Extracción híbrida instantánea y OCR de alta precisión para documentos escaneados
Búsqueda Vectorial	Azure AI Search	Indexación semántica y recuperación de expedientes legales
Almacenamiento de Archivos	Azure Blob Storage (StorageV2)	Repositorio documental cifrado en reposo para expedientes y contratos
Inteligencia de Negocio	Microsoft Power BI (SQL Views + REST Feed)	Tableros ejecutivos, medición de SLAs, tiempos de ciclo y cuellos de botella
Panel de Administración	SQLAdmin	Gestión directa de entidades ORM para administradores de plataforma
Infraestructura como Código	Azure ARM (`azuredeploy.json`) + Bicep (`main.bicep`)	Despliegue automatizado "Click-to-Go" *zero-touch* en la nube
DevSecOps & CI/CD	GitHub Actions (Ruff, Semgrep, Bandit, pip-audit, Pytest)	Pipeline de verificación estática, OWASP Top 10, CVEs y despliegue a Azure

🏛️ Arquitectura del Repositorio

```
Nos-vamos-a-graduar/
├── alembic/                        # 🗄️ Control de versiones de Base de Datos
│   ├── env.py                      # Conexión y metadatos de modelos SQLAlchemy
│   └── versions/                   # Migraciones de esquema
├── app/                            # 📁 NÚCLEO DE LA APLICACIÓN BACKEND & WEB
│   ├── main.py                     # Fábrica de aplicación FastAPI, routers y middlewares
│   ├── admin.py                    # Panel administrativo SQLAdmin protegido (HU-15)
│   ├── api/v1/                     # 🌐 Controladores HTTP y enrutamiento modular:
│   │   ├── router.py               # Enrutador central con dependencias de seguridad
│   │   └── endpoints/              # Endpoints especializados:
│   │       ├── auth.py             # Autenticación Entra ID, configuración MSAL y sesión
│   │       ├── frontend.py         # Enrutador de vistas Jinja2 (Dashboard, Tareas, etc.)
│   │       ├── solicitudes.py      # Creación y gestión de expedientes (HU-01, HU-04)
│   │       ├── workflows.py        # Bandejas de trabajo (HU-02) y etapas BPM (HU-16)
│   │       ├── documents.py        # Carga, descarga y versiones (HU-08, HU-09, HU-17)
│   │       ├── reviews.py          # Dictámenes y comentarios en hilo (HU-05, HU-06, HU-07)
│   │       ├── signatures.py       # Aprobaciones y firmas electrónicas (HU-10, HU-11)
│   │       ├── ai_legal.py         # Asistencia con Azure OpenAI (HU-14)
│   │       ├── analytics.py        # Feeds Power BI, cálculo de KPIs y exportación CSV
│   │       ├── audit.py            # Trazabilidad forense (HU-12)
│   │       ├── notifications.py    # Avisos y alertas de SLA (HU-13)
│   │       ├── users.py            # Gestión y roles de usuarios (HU-15)
│   │       ├── organizations.py    # Gestión de organizaciones
│   │       └── search.py           # Búsqueda semántica con Azure Search
│   ├── core/                       # ⚙️ Configuraciones transversales y seguridad:
│   │   ├── config.py               # Pydantic BaseSettings (.env centralizado)
│   │   ├── database.py             # SQLAlchemy SessionLocal, Base y dependencias
│   │   ├── auth_guard.py           # Validación de tokens JWT y API Key
│   │   ├── roles_guard.py          # Control de acceso basado en roles (@requiere_roles)
│   │   ├── rate_limiter.py         # Rate Limiting por token bucket
│   │   ├── security.py             # Cifrado simétrico AES-256-GCM y hashing SHA-256
│   │   └── audit_helper.py         # Helper obligatorio de trazabilidad forense (HU-12)
│   ├── models/                     # 🗃️ Modelos de Base de Datos relacionales (SQLAlchemy)
│   │   ├── user.py                 # Usuario y catálogo oficial RolUsuario
│   │   ├── solicitud.py            # SolicitudRevision y EstadoSolicitud
│   │   ├── document.py             # Documento
│   │   ├── version.py              # VersionDocumento (inmutabilidad y hash)
│   │   ├── comment.py              # Comentario (hilos de respuesta anidados)
│   │   ├── revision.py             # RevisionJuridica y Dictamen
│   │   ├── audit.py                # Auditoria (bitácora forense inmutable)
│   │   ├── workflow.py             # Modelos de flujo de trabajo BPM y etapas
│   │   ├── tenant.py               # Modelo multi-organización
│   │   └── extras.py               # Firma, Notificacion
│   ├── schemas/                    # 📐 Esquemas de validación y serialización Pydantic
│   ├── services/                   # 🧠 Lógica de negocio pesada e integraciones cloud:
│   │   ├── workflow_engine.py      # Motor BPM y asignación automática por carga (HU-03)
│   │   ├── document_extractor.py   # Extractor híbrido digital + OCR con PyMuPDF
│   │   ├── azure_blob.py           # Repositorio Azure Blob Storage (HU-17)
│   │   ├── azure_doc_intel.py      # OCR y extracción tabular con Document Intelligence (HU-14)
│   │   ├── azure_openai.py         # Análisis con Azure OpenAI y LangChain (HU-14)
│   │   └── azure_search.py         # Búsqueda vectorial
```

```
│   ├── static/                    # 🎨 Archivos estáticos (CSS, JS local MSAL, iconos)
│   └── templates/                 # 🖥️ Plantillas Jinja2 (13 pantallas del portal)
├── docs/                          # 📚 CENTRO DE DOCUMENTACIÓN OFICIAL DEL SISTEMA
│   ├── INDEX.md                   # Índice maestro de documentación
│   ├── MANUAL_USUARIO_Y_ROLES.md  # Manual operativo para usuarios finales y roles
│   ├── ARQUITECTURA_SISTEMA.md    # Especificación arquitectónica y diagramas C4
│   ├── MATRIZ_HISTORIAS_USUARIO.md # Trazabilidad formal HU-01 a HU-17 y criterios
│   ├── API_REFERENCE.md           # Referencia técnica de endpoints RESTful v1
│   ├── SEGURIDAD_Y_DEVSECOPS.md   # Guía de seguridad OWASP y DevSecOps
│   └── GUIA_ENTREGA_CLIENTE_AZURE.md# Procedimiento de despliegue Click-to-Go en Azure
├── infra/                         # ☁️ Plantillas de Infraestructura como Código (IaC)
│   ├── azuredeploy.json           # Plantilla ARM para despliegue automatizado
│   └── main.bicep                 # Especificación moderna Bicep para Azure
├── powerbi/                       # 📊 Recursos para Microsoft Power BI
│   ├── README_POWERBI.md          # Manual de conexión y medidas DAX
│   └── vistas_powerbi.sql         # Vistas SQL optimizadas para KPIs y SLA
├── scripts/                       # 🛠️ Scripts de utilidad y automatización
│   ├── deploy_client_infra.ps1    # Despliegue automatizado en Azure con PowerShell
│   ├── deploy_client_infra.sh     # Despliegue automatizado en Azure con Bash
│   └── security_audit.py          # Auditoría local de seguridad
├── tests/                         # 🧪 Suite de pruebas automáticas (58 pruebas)
│   ├── conftest.py                # Fixtures de BD aislada y cliente de pruebas
│   ├── test_access_control.py     # RBAC, roles, multi-tenant y sesiones
│   ├── test_ai_legal.py           # Endpoints y mocks de Azure OpenAI
│   ├── test_analytics.py          # KPIs, feeds Power BI y exportación CSV
│   ├── test_audit.py              # Trazabilidad y bitácora forense (HU-12)
│   ├── test_crypto.py             # Cifrado AES-256-GCM e integridad SHA-256
│   ├── test_emergency_troubleshoot.py # Resiliencia, rate limiter y extractor OCR
│   ├── test_main.py               # Raíz, sondas de salud y cabeceras OWASP
│   ├── test_solicitudes.py        # Ciclo de vida y validación de expedientes (HU-01)
│   ├── test_users.py              # Aprovisionamiento y perfil de usuarios
│   └── test_workflows.py          # Bandejas de trabajo e inbox (HU-02)
├── .github/workflows/             # 🤖 Pipeline DevSecOps (Ruff, Semgrep, Bandit, pip-audit, Pytest)
├── docker-compose.dev.yml         # 🐳 Contenedor local para PostgreSQL 16
├── docker-compose.yml             # 🐳 Contenedor de aplicación y base de datos
├── Dockerfile                     # 🐳 Imagen Docker lista para producción
├── .env.example                   # 📋 Plantilla comentada de variables de entorno
└── pyproject.toml                 # ⚙️ Configuración centralizada de herramientas de calidad
```

🚫 Reglas de Oro para Desarrolladores

1. **Trazabilidad Obligatoria (HU-12 / RN-010):**
   - Toda función o endpoint que cree, modifique, apruebe o elimine un registro DEBE invocar `registrar_auditoria(db, usuario, accion=..., tipo_documento=..., id_documento=...)`. La trazabilidad jamás se omite.
2. **Supervisión Humana en IA (HU-14 / RN-008):**
   - La inteligencia artificial es un **asistente de apoyo técnico**. Las recomendaciones de Azure OpenAI jamás autorizan o rechazan automáticamente un contrato sin la intervención y dictamen explícito de un abogado o aprobador facultado.

3. **Inmutabilidad de Versiones (HU-08 / RN-004):**
   - Una versión de documento enviada a revisión o formalizada en firma electrónica no puede modificarse. Todo nuevo ajuste debe generar una versión incremental inmutable (`VersionDocumento`) con su propio hash SHA-256.
4. **Protección de Datos Sensibles (RNF-003):**
   - Todo campo clasificado como confidencial en base de datos debe utilizar el módulo criptográfico centralizado:

     ```
     from app.core.security import encrypt_text, decrypt_text
     ```
5. **Cero Secretos en Código:**
   - NUNCA agregues API Keys, secretos de cliente o contraseñas al código. Utiliza variables de entorno en el archivo local `.env` (ignorado en `.gitignore`) mapeadas en `app/core/config.py` .
6. **Control de Acceso Estricto (RBAC):**
   - Protege los endpoints utilizando siempre el decorador de roles con los perfiles oficiales de Microsoft Entra ID:

     ```
     @router.post("/ruta")
     def mi_endpoint(usuario: dict = Depends(requiere_roles("REVISOR_JURIDICO", "ADMINISTRADOR_PLATAFORMA"))):
         ...
     ```

🌿 Flujo de Trabajo en Git

Para cualquier mejora o mantenimiento evolutivo:

1. Actualizar tu rama local con `main`

```
git checkout main
git pull origin main
```

2. Crear una rama vinculada a una Historia de Usuario o Tarea

Nombra tu rama utilizando el identificador correspondiente:

```
git checkout -b feature/HU-14-optimizacion-tokens
# Ejemplos:
# git checkout -b fix/HU-08-inmutabilidad-hash
# git checkout -b docs/actualizacion-manual-roles
```

3. Convención de Commits Semánticos

Cada commit debe indicar la naturaleza del cambio y la HU asociada:

```
git commit -m "feat(HU-14): optimizar cuota de tokens con reasoning bajo en Azure OpenAI"
git commit -m "fix(HU-12): registrar direccion IP de origen en auditoria forense"
git commit -m "docs: actualizar manual de usuario y matriz de historias"
```

💻 Entorno de Desarrollo Local y Verificación

1. Prerrequisitos

- Python 3.11+
- Docker Desktop (opcional, para levantar PostgreSQL local)
- Git

2. Puesta en Marcha Rápida

```
# 1. Crear y activar entorno virtual
python -m venv .venv
# En Windows:
.venv\Scripts\activate
# En Linux/macOS:
# source .venv/bin/activate

# 2. Instalar dependencias
pip install -r requirements.txt

# 3. Configurar variables de entorno
cp .env.example .env
# Completa tus credenciales de base de datos y Azure en .env

# 4. Levantar base de datos PostgreSQL local en Docker (opcional)
docker compose -f docker-compose.dev.yml up -d

# 5. Aplicar migraciones con Alembic
alembic upgrade head

# 6. Iniciar el servidor de desarrollo
python main.py
```

Puntos de acceso en el navegador:

- **Portal de Acceso:** http://localhost:8000/login
- **Dashboard:** http://localhost:8000/api/v1/app/dashboard
- **Documentación Swagger:** http://localhost:8000/docs
- **Panel Administrativo:** http://localhost:8000/admin

🧪 Validación Obligatoria Pre-Push (DevSecOps)

Antes de abrir un Pull Request hacia `main`, debes ejecutar y pasar localmente los **5 filtros de calidad y seguridad**:

```
# 1. Calidad y formato de código (Ruff)
ruff check .

# 2. Análisis de seguridad de patrones OWASP Top 10 (Semgrep)
semgrep scan --config "p/owasp-top-ten" --config "p/python" --metrics=off --error app/

# 3. SAST de vulnerabilidades en código Python (Bandit)
bandit -r . -x ./.venv,./tests -ll

# 4. Auditoría de dependencias conocidas contra CVEs (pip-audit)
pip-audit -r requirements.txt

# 5. Suite completa de 58 pruebas automáticas (Pytest)
python -m pytest -v
```

Cuando todos los filtros pasen en verde (0 errores), sube tu rama y genera tu **Pull Request** para su revisión e integración continua en Azure App Service.